IUBAT – Internation Univesity of Business Agriculture & Technology
Founded 1991 by Md. Alimullah Miyan

Course Information

Course Name: Programming Languages and Structures
Course Code: CSC 461

Final Assignment

LLM-Inspired Recursive Descent Parser

Submitted By

Name	Id
Aononto Jahan Junnurain	21103030
Md Rasheduzzaman Rashead	22203079
Kanij Momota	22203002

Submitted To

Dr. Ishtiak Al Mamoon
Associate Professor, CSE, IUBAT

Date of Submission: 07-01-2026

Table of Contents

1. Introduction 3
2. EBNF Grammar Design 3
   2.1 Grammar Specification 3
   2.2 Grammar Explanation 3
3. Recursive Descent Parser Implementation 4
4. LLM-Inspired Adaptability and Design Insight 4
5. Error Handling and Recovery Mechanism 4
6. Test Case Analysis 5
   6.1 Provided Test Cases 5
7. Codes & Output 5
   7.1 Codes 5
   7.2 Output 10
8. How to run this project 10
9. KPA Justiication 11

LLM-Inspired Recursive Descent Parser

1. Introduction

As part of the course Programming Languages and Structures (CSC 461), this assignment describes the design and implementation of a Recursive Descent Parser influenced by the Large Language Model (LLM). This work's main goal is to use parsing and grammar formation tools to examine the structural features of programming languages.

In contrast to conventional deterministic parsers, this solution uses recovery and adaptation methods that draw inspiration from contemporary LLM-based language comprehension systems. The parser accommodates nested structures, handles arithmetic-like expressions, preserves operator precedence, and uses heuristic-based recovery to manage unclear or incorrect inputs.

2. EBNF Grammar Design

2.1 Grammar Specification

```
expression = term , { ("+" | "-") , term } ;
term       = factor , { ("*" | "/" | "%") , factor } ;
factor     = number
             | "(", expression , ")"
             | "-" , factor
             | identifier ;

number     =  digit , { digit } , [ "." , digit , { digit } ] ;

identifier  =  letter , { letter | digit } ;

digit      = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

letter     = "a" | "b" | "c" | ... | "z"
             | "A" | "B" | "C" | ... | "Z" ;
```

2.2 Grammar Explanation

In order to maintain operator precedence and associativity, the grammar is structured hierarchically. Multiplicative operators are addressed at the term level, whereas additive operators are managed at the expression level. Numerical literals, identifiers, nested expressions, and unary operators are all supported by the factor rule. LLM-inspired adaptability is made possible by the addition of identifiers, which enable expansion toward non-numeric tokens

3. Recursive Descent Parser Implementation

Every non-terminal symbol in the language has a corresponding parsing function in the top-down recursive descent method used to create the parser. To handle terms and factors, the parsing process descends recursively from the expression level.

Function hierarchy naturally enforces operator precedence, and recursion makes it possible to evaluate nested expressions. Future additions, including predictive parsing or integration with neural language models, are made possible by the implementation's modular design.

4. LLM-Inspired Adaptability and Design Insight

Conventional recursive descent parsers are deterministic and stop running when they come upon incorrect syntax. On the other hand, by accepting ambiguity and recovering from faulty input, this parser integrates LLM-inspired flexibility.

Instead of failing right away, heuristic prediction is used to handle unknown identifiers. To maintain parsing continuity, missing delimiters like parentheses are automatically fixed. Abrupt termination is replaced by informative diagnostic signals, which mirror contemporary NLP-inspired validation techniques frequently employed in big language models.

5. Error Handling and Recovery Mechanism

To improve the dependability of the parser, robust error handling is integrated. Token location information is one of the useful diagnostic signals that are provided by custom parsing exceptions. Errors caused by division by zero and modulo by zero are specifically identified and reported.

When parentheses are absent, the parser makes an effort to recover gracefully by assuming implicit closure. To avoid silent misreading of expressions, trailing or unexpected tokens are also identified. These mechanisms exhibit LLM-inspired robustness to distorted input and enhance fault tolerance.

6. Test Case Analysis

6.1 Provided Test Cases

Test Case 1:
Input: (((3 + 2) * ((7 - (4 + 1)) + 9)) / 5) - 3
Output: 8
Description: Validates deep recursion, nested expressions, and operator precedence.

Test Case 2:
Input: --5 + ((3 * (2 + (1 - 4))) / 2)
Output: 3.5
Description: Tests unary operator handling and nested arithmetic.

Test Case 3:
Input: 7 + alpha * (3 - 1)
Output: 17
Description: Demonstrates LLM-inspired handling of unknown identifiers.

6.2 Additional Investigation

Malformed expressions, missing parentheses, division by zero, and trailing tokens were used in further tests. The capacity of the parser to recover gracefully and offer insightful feedback without abrupt termination was verified by these tests.

7. Codes & Output

7.1 Codes

```
# Project Name: LLM Inspired Recursive Descent Parser
# Course Name: Programming Languages & Structures (CSC 461)
# This project implements a recursive descent parser that evaluates
# arithmetic expressions while demonstrating adaptability inspired by
# large language models (LLMs). The parser handles operator precedence,
# unary operators, and includes robust error handling and recovery
# mechanisms. It also investigates edge cases to ensure comprehensive
# parsing capabilities.
```

```
# Exception class for parsing errors
class ParseError(Exception):
    """Custom exception for syntax and parsing errors"""
    pass
# COnverting input string into streams of tokens
def tokenize(expression):
    tokens = []
    i = 0

    while i < len(expression):
        char = expression[i]

        # Ignoring thr whitespace
        if char.isspace():
            i += 1
            continue

        # Handling numeric literals (integers and floats)
        if char.isdigit() or char == '.':
            num = char
            i += 1
            while i < len(expression) and (
                expression[i].isdigit() or expression[i] == '.'
            ):
                num += expression[i]
                i += 1
            tokens.append(num)
            continue

        # Handling identifiers (alphabetic tokens)
        if char.isalpha():
            name = char
            i += 1
            while i < len(expression) and expression[i].isalnum():
                name += expression[i]
                i += 1
            tokens.append(name)
            continue

        # Operators and parentheses
        tokens.append(char)
        i += 1

    return tokens




# PARSER CLASS (Recursive Descent Implementation)
# Grammar:
```

```
# expression → term { (+|-) term }
# term       → factor { (*|/|%) factor }
# factor     → number | (expression) | -factor | identifier

class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos = 0  # Current token position

    # Return current token
    def current_token(self):
        if self.pos < len(self.tokens):
            return self.tokens[self.pos]
        return None

    # Consume expected token
    def eat(self, token):
        if self.current_token() == token:
            self.pos += 1
        else:
            raise ParseError(
                f"Expected '{token}', found '{self.current_token()}' "
                f"at position {self.pos}"
            )


    # EXPRESSION LEVEL (+ and -)
    def parse_expression(self):
        value = self.parse_term()

        while self.current_token() in ('+', '-'):
            op = self.current_token()
            self.eat(op)

            if op == '+':
                value += self.parse_term()
            else:
                value -= self.parse_term()

        return value


    # TERM LEVEL (*, /, %)

    def parse_term(self):
        value = self.parse_factor()

        while self.current_token() in ('*', '/', '%'):
            op = self.current_token()
            self.eat(op)
            if op == '*':
```

```
            value *= self.parse_factor()
        elif op == '/':
            divisor = self.parse_factor()
            if divisor == 0:
                raise ParseError("Division by zero detected")
            value /= divisor
        elif op == '%':
            divisor = self.parse_factor()
            if divisor == 0:
                raise ParseError("Modulo by zero detected")
            value %= divisor

    return value

# FACTOR LEVEL

def parse_factor(self):
    token = self.current_token()

    # Unary minus handling
    if token == '-':
        self.eat('-')
        return -self.parse_factor()

    # Unexpected end of input
    if token is None:
        raise ParseError("Unexpected end of input")

    # Numeric literal validation
    if token.count('.') <= 1 and token.replace('.', '', 1).isdigit():
        self.eat(token)
        return float(token)

    # Identifier handling (LLM-inspired prediction)
    if token.isalpha():
        predicted_value = len(token)  # Simple heuristic
        print(
            f"Warning: Unknown identifier '{token}' "
            f"predicted as {predicted_value}"
        )
        self.eat(token)
        return float(predicted_value)

    # Parenthesized expression
    if token == '(':
        self.eat('(')
        value = self.parse_expression()

        # Graceful recovery for missing ')'
        if self.current_token() == ')':
            self.eat(')')
```

```
            else:
                print("Warning: Missing ')', auto-corrected")

            return value

        # Invalid token
        raise ParseError(
            f"Invalid token '{token}' at position {self.pos}"
        )



# EVALUATION FUNCTION

def evaluate(expression):
    tokens = tokenize(expression)
    parser = Parser(tokens)
    result = parser.parse_expression()

    # Detect trailing tokens after valid expression
    if parser.current_token() is not None:
        raise ParseError(
            f"Unexpected token '{parser.current_token()}' "
            f"after complete expression"
        )

    return result



# MAIN PROGRAM (USER INPUT MODE)

if __name__ == "__main__":

    print("**************************************************")
    print(" LLM-Inspired Recursive Descent Parser (CSC 461)")
    print(" Type 'exit' to terminate the program")
    print("**************************************************\n")


 history = []  # Store previous expressions and results

    while True:
        user_input = input("Enter expression: ")

        # Exit condition
        if user_input.lower() == "exit":
            print("\nProgram terminated.")
            break

        try:
```

```
        result = evaluate(user_input)
        history.append((user_input, result))
        print("Result:", result)

    except ParseError as error:
        print("Parsing Error:", error)
        print("Hint: Check syntax, operators, or parentheses.")

# Display expression history
print("\nExpression History:")
for expr, res in history:
    print(f"{expr} = {res}")
```

7.2 Output

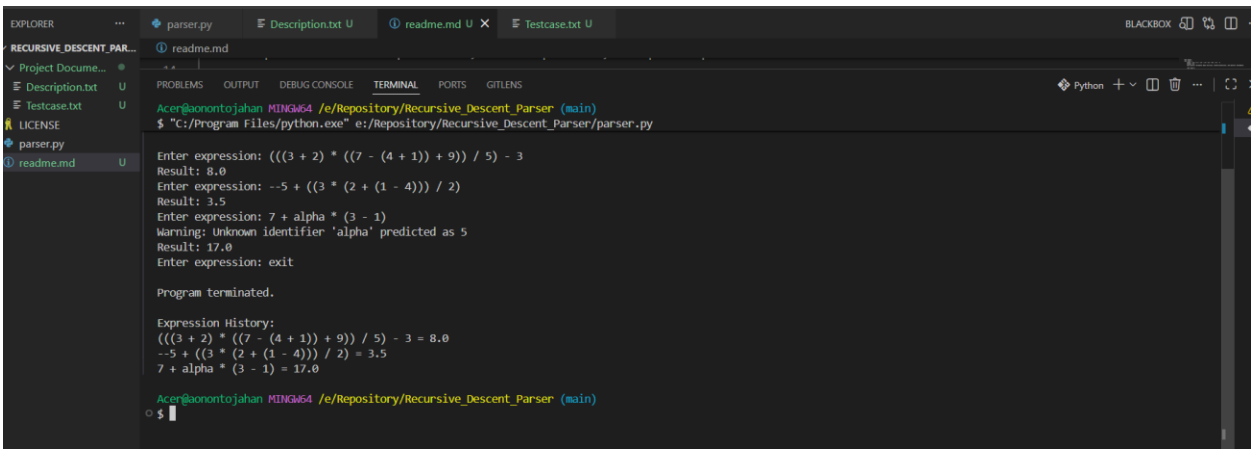

Figure 1: LLM-Inspired Recursive Descent Parser

8. How to run this project

This project name is LLM Inspired Recursive Descent Parser
How to run this project:

- -Install Python 3.10.*
- -Setup python
- -Open the folder in a python IDE
- -Run the File name parser.py

10. KPA Justiication

IUBAT – Internation Univesity of Business Agriculture & Technology
Founded 1991 by Md. Alimullah Miyan

Department of Computer Science and Engineering
Bachelor of Computer Science and Engineering (BCSE)
Course (Theory/Lab) along with
Complex Engineering Problem (CEP) and Complex Engineering Activities (CEA)

Course Code and Title: CSC 461 Programming Logic Structure
Semester: Fall 2025
Student Name: Aononto Jahan Junnurain
Student ID: 21103030
Course Instructor Name: Dr. Ishtiak Al Mamoon

Name of CEA	Explanation	
CO 4 **Course Outcome**	This assignment demonstrates investigation and analysis of programming language structures through grammar formulation and recursive descent parsing. The comparison between deterministic parsing and adaptive LLM-inspired strategies reflects the chronological evolution of programming language processing techniques.	
P1: **Depth of Knowledge**	**K3**	Evidenced through the design of an extensible EBNF grammar, modular parser architecture, and operator precedence handling.
	K4	Demonstrated by experimentation with ambiguous inputs, unknown identifiers, and error recovery mechanisms inspired by LLM behavior.
P3: **Depth of Analysis**	The problem requires abstract reasoning beyond textbook parsing techniques. Conflicting requirements between strict syntax enforcement and adaptive recovery necessitate original modeling decisions, addressing complexity levels P3	